Methods Implemented: SJF, RR, SRT

SJF RESULTS :

```
SJF Scheduling Timeline:
p1: [0-3]
p2: [3-9]
p5: [9-11]
p3: [11-15]
p4: [15-20]

Turnaround Times:
p1: 3
p2: 7
p3: 11
p4: 14
p5: 3

Average Turnaround Time: 7.6
```

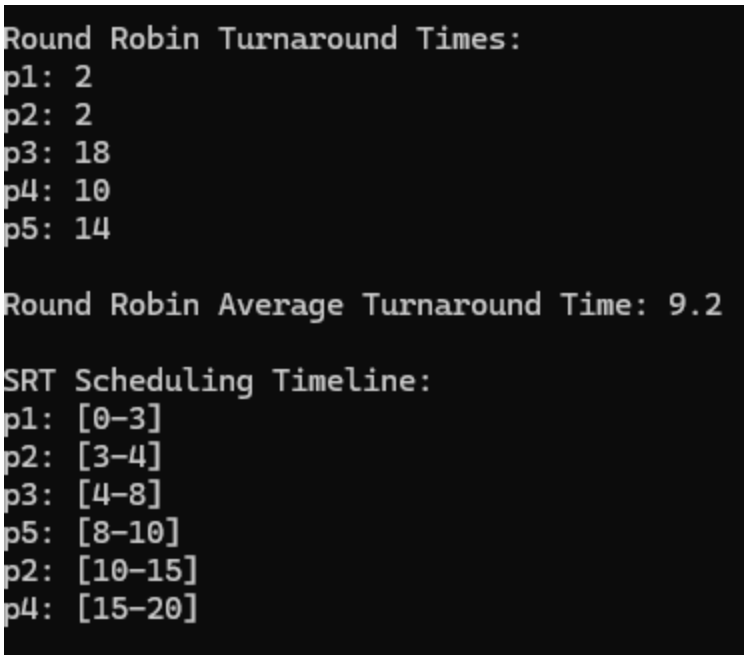
SRT:

```
SRT Scheduling Timeline:
p1: [0-3]
p2: [3-4]
p3: [4-8]
p5: [8-10]
p2: [10-15]
p4: [15-20]

SRT Turnaround Times:
p1: 3
p2: 13
p3: 4
p4: 14
p5: 2

SRT Average Turnaround Time: 7.2
```

Round Robin:



Overall Results: Of the 3 algorithms, SRT had the lowest average turnaround time; second was SJF, and lastly round robin

Design Choices:

I used Python to implement the 3 algorithms. I created a Process class to store the process ID, arrival time, and CPU burst time. To organize the processes, I used lists and dictionaries to track the remaining CPU time and completion time. For Round Robin, I used a deque for the ready queue to keep it in FIFO order.

For SJF, the process with the shortest CPU burst from the ready list ran until it finished. For SRT, I selected the process with the shortest remaining CPU time. It was important to check the

available processes to determine if another process should run. If two processes had the same shortest remaining time, the current process continued running. For Round Robin, I used a FIFO ready queue with a time quantum of 2 ms.

Preemption/Quantum Handling:

1. **SJF:** Non-preemptive, meaning once a process starts, it runs until it finishes.
2. **SRT:** Preemptive, meaning a process can be interrupted if another process arrives with a shorter remaining CPU time.
3. **Round Robin:** Preemptive, meaning each process can run for up to 2 ms before moving to the back of the ready queue if it is not finished.

Takeaways:

I learned that each scheduling algorithm affects how fast processes finish. While running the code, it was easier to understand how preemption helps shorter processes finish sooner since they don't have to wait for longer processes to finish.